

Mitigating Layer-2 Attacks Using Native XDP and its Performance Implications

Abhyuday K Hegde¹ and Vidith Venkatesha Murthy²

¹Roll No: 241CS201.

²Roll No: 241CS263.

Contributing authors: akh.241cs201@nitk.edu.in;
vidithvenkateshamurthy.241cs263@nitk.edu.in;

Abstract

Layer 2 attacks like ARP Spoofing and MAC Flooding pose significant threat to Local Area Networks (LANS) by allowing attackers to manipulate IP-MAC binding and disrupt the normal packet forwarding. A key challenge in mitigating these attacks is the inherent insecurity of ARP Protocol which lacks authentication and allows unsolicited updates to IP-MAC binding. Existing solutions such as Dynamic ARP Inspection (DAI) rely heavily on specialized switch hardware and DHCP snooping tables which limit their applicability. Host-based defenses using iptables operate later in the network stack. Intrusion detection systems can identify anomalies but are typically reactive and do not prevent malicious traffic in real time. As a result, there is a need for a high-performance solution that can operate early in the packet processing pipeline. This paper proposes an XDP based framework for early detection and mitigation of ARP spoofing and MAC flooding attacks in Linux based networking environments. The system leverages eBPF maps for dynamic IP-MAC binding derived from DHCP snooping and integrates a user-space control plane for adaptive validation. By performing packet inspection at the driver level, the proposed approach enables real time enforcement with minimal latency and overhead. The primary objective of this paper is to design and implement lightweight detection and mitigation mechanisms for Layer 2 attacks specifically ARP spoofing and MAC flooding using native XDP.

Keywords: ARP Spoofing, MAC Flooding, XDP, DAI

1 Introduction

Local Area Networks (LANs) form the foundation of modern enterprise, campus and data center environments, where devices within the same broadcast domain rely on Layer-2 protocols for efficient packet forwarding and address resolution. The Address Resolution Protocol (ARP), standardized in RFC 826 is universally used to resolve IP addresses to their corresponding MAC addresses, enabling communication at the Ethernet layer. However, ARP was designed without security considerations and operates without authentication or verification. Any host on a network segment can transmit ARP replies and receiving devices will update their ARP caches upon receipt, regardless of whether the response was solicited. This design limitation makes ARP highly susceptible to exploitation [1–3]. In parallel, Ethernet switches maintain Content Addressable Memory (CAM) tables to map MAC addresses to physical ports for efficient unicast forwarding. When these tables are exhausted, switches fall back to broadcast behavior, forwarding frames to all ports. Attackers exploit this through MAC flooding, injecting frames with randomized source MAC addresses at high rates until the CAM table overflows, thereby exposing traffic across the network [4]. Together, ARP spoofing and MAC flooding make up critical Layer-2 attack vectors capable of enabling man-in-the-middle interception, traffic manipulation and denial of service conditions.

Mitigating these attacks presents several challenges. The fundamental issue lies in the stateless and unauthenticated nature of ARP as it allows unsolicited and potentially malicious updates to IP–MAC mappings. At the same time, legitimate network operations introduce similar patterns of behavior which complicates detection. Gratuitous ARP messages are commonly used for failover mechanisms and IP conflict detection, while DHCP based reassignment leads to valid changes in IP–MAC bindings over time. Modern mobile devices further introduce variability through MAC address randomization [5], causing frequent and legitimate mapping updates. These factors make it difficult to distinguish between genuine and malicious activity without introducing false positives. Additionally, MAC flooding attacks generate traffic at very high rates, overwhelming traditional software based inspection mechanisms that operate deeper in the kernel networking stack, leading to increased latency and degraded performance [4]. In virtualized and cloud-native environments, where software defined networking replaces dedicated hardware switches, these challenges are further amplified due to the absence of hardware level protections and the presence of multi-tenant broadcast domains [6].

Existing approaches to mitigate ARP spoofing and related Layer-2 attacks exhibit significant limitations. On most of the enterprise switches, Dynamic ARP Inspection (DAI) is widely deployed.[7] It validates ARP packets against DHCP snooping tables but is tightly coupled to specialized switch hardware, limiting its applicability in software defined environments. Protection for ARP spoofing with static ARP entries may increase the administrative overhead and requires higher processing power. [8]. Packet filtering frameworks like iptables and nftables operate at later stages of the network stack [9]. This makes them less effective against high rate attacks due to increased processing overhead. Intrusion detection systems such as Snort and Suricata can identify

anomalous patterns but are inherently reactive and do not prevent malicious packets from entering the network. [10]. Software defined networking approaches introduce programmability but often come with additional architectural complexity and deployment constraints [6]. As a result, there remains a gap for a solution that combines early packet interception, high performance and adaptability to dynamic network behavior.

This work addresses this gap by proposing an eBPF-based framework that leverages the eXpress Data Path (XDP) to perform early stage packet filtering at the network interface level. The system intercepts ARP and Ethernet frames at the earliest point in the packet processing pipeline, validates IP-MAC bindings against dynamically maintained trust tables and drops malicious packets before they propagate into the kernel networking stack. By integrating a user-space control plane with DHCP aware synchronization and adaptive verification, the proposed approach maintains robustness against attacks while accommodating legitimate network changes.

The objectives of this work are two fold: to design and implement lightweight XDP-based mechanisms for detecting and mitigating ARP spoofing and MAC flooding attacks and to evaluate the system’s performance in terms of latency, throughput and resource utilization.

With the introduction covered in Section 1, the literature overview that compares multiple proposed approaches, along with their advantages and disadvantages, is briefly discussed in Section 2, with subsections. In Section 3, the paper elaborates on the proposed methodology of a unified XDP-based solution to mitigate ARP Spoofing and MAC Flooding attacks also providing their algorithms. The implementation of these algorithms is done in Section 4. This is followed by a description on the experiment environment setup and the performance analysis of the code in Section 5. Finally, the paper concludes with Section 6 briefing the current accomplishment and future scope of the paper.

2 Literature Overview

This section reviews various proposed solutions to mitigate ARP spoofing and MAC flooding attacks. Subsection 2.1 presents a broadcast-based verification approach for ARP validation, followed by Subsection 2.2 which examines machine learning based techniques for detecting anomalous network behavior. Subsection 2.3 discusses a database driven method for verifying IP-MAC bindings using persistent storage, with Subsection 2.4 evaluating port security mechanisms designed to prevent MAC flooding at the switch level. Finally, subsection 2.5 provides a comparative analysis of these approaches, highlighting their limitations and motivating the need for a unified, low latency and high performance solution.

2.1 Broadcast-Based Verification Approach

Mary et al. [1] proposed a broadcast-based verification approach that attempts to mitigate ARP spoofing by actively validating incoming ARP packets through secondary network queries. In this model, upon receiving an initial ARP packet, the detection system broadcasts a new ARP request to query the MAC address associated with the claimed IP address. This is illustrated in Fig.1. The system operates on

the assumption that the subsequent ARP reply represents the ground truth, which is then compared against the MAC address of the original packet.

While this methodology offers a straightforward implementation without requiring modifications to the underlying ARP protocol, it relies on an unauthenticated secondary response as an absolute truth leaving the system highly vulnerable at times. An active attacker can outpace the legitimate host in responding to the secondary broadcast, effectively bypassing the detection mechanism. Furthermore, triggering a network wide broadcast for every received ARP packet introduces severe network overhead, posing a risk of self-inflicted broadcast storms and state exhaustion during a high volume spoofing attack. An evaluation of its operational advantages and limitations detailed in Table 1.

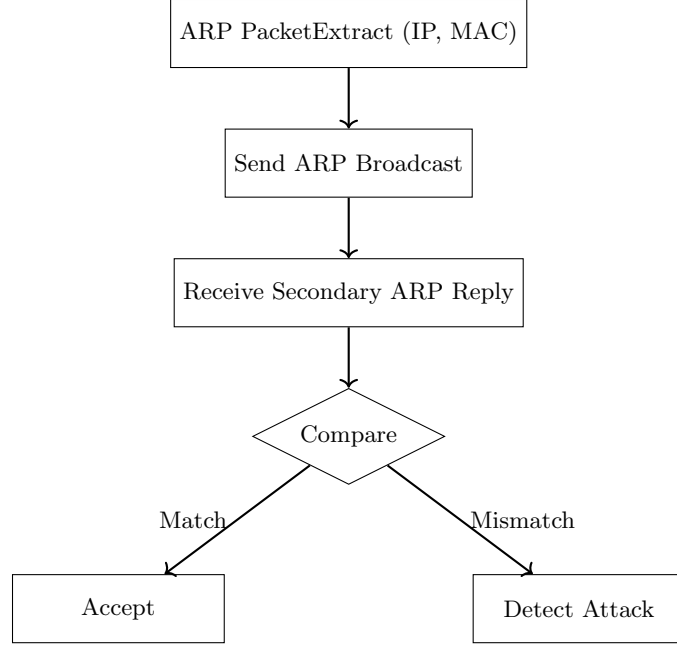


Fig. 1: ARP Spoofing Detection Flowchart

2.2 Machine Learning-Based Approach

Machine learning-based approaches leverage statistical models, such as Support Vector Machines (SVM) and Multi-Layer Perceptrons (MLP), to detect anomalous network behavior indicative of ARP spoofing. As demonstrated by Praveena and Sudha [11], these models can be trained to classify network traffic as normal or malicious, with deep learning models like MLP reportedly achieving high theoretical accuracies (up to 97%). In this approach, raw traffic data first undergoes feature extraction to isolate key network attributes. These extracted features are then fed into

Pros	Cons
No protocol modification	High network overhead (extra broadcasts)
Simple implementation	Increased latency
	Not scalable
	Reactive detection approach

Table 1: Pros and Cons of Broadcast-Based Verification Approach

the trained ML Model for analysis. Based on this evaluation, the system performs a classification wherein, normal traffic is permitted to pass, while any identified Attack triggers an immediate Alert. The operational flow is illustrated in Fig. 2.

However, a primary challenge faced by this method is dataset relevance. Many ML based intrusion detection studies rely on generalized Network Intrusion Detection System (NIDS) datasets containing Layer 3 and Layer 4 features (e.g., TCP connections, login attempts) that do not accurately represent Layer 2 ARP packet structures [2]. Furthermore, deploying heavy computational models like SVM or MLP for per-packet inspection introduces significant latency, making real time, packet-level mitigation difficult in high speed enterprise networks. The advantages and disadvantages of this approach are summarized in Table 2.

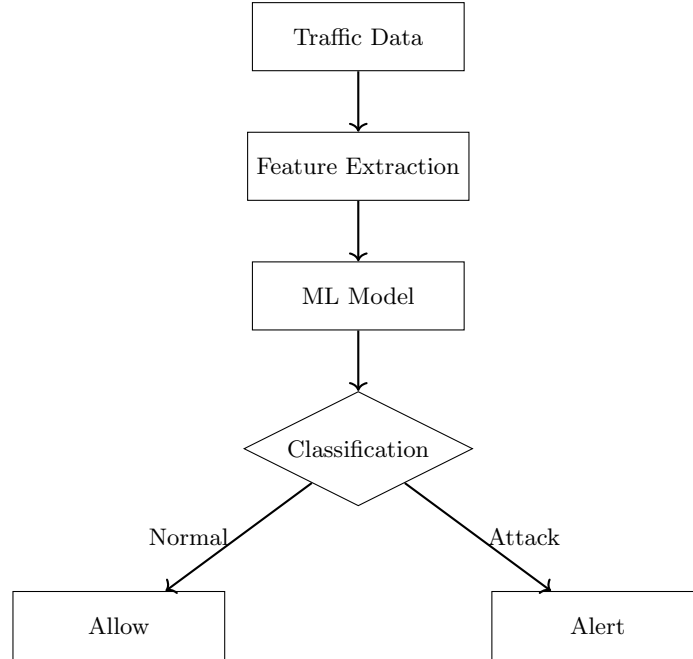


Fig. 2: Machine Learning-Based Traffic Classification Flowchart

Pros	Cons
High detection accuracy	High computational overhead (latency for real-time traffic)
Capable of detecting complex, non linear anomaly patterns	Heavy reliance on high quality, Layer-2 specific training data
Can adapt to varying network baselines over time	Difficult to achieve per-packet, real time blocking
	Susceptible to false positives during normal network topology

Table 2: Pros and Cons of Machine Learning-Based Approach

2.3 Database-Driven Approach

Galal et al. [3] proposed an architecture utilizing an external MySQL database synchronized with the network switch. This approach relies on maintaining a persistent, centralized repository of authorized IP MAC bindings to validate incoming ARP packets. Upon receiving an ARP packet, the system extracts the IP and MAC addresses, computes a cryptographic hash (MD5) and queries the database to verify the mapping against stored credentials. as shown in Fig. 3. While maintaining a persistent database successfully solves the issue of volatile ARP caches clearing after timeouts or switch reboots, querying an external, software based database for routine Layer 2 data-link operations increases latency. The advantages and disadvantages of this approach is summarized in Table 3.

Pros	Cons
Persistent Detection	Database lookup overhead
High Reliability	Scalability issues
Detects Multiple Anomalies	Added latency
	Requires extra infrastructure

Table 3: Pros and Cons of Database-Driven Approach

2.4 Port Security for MAC Flooding

. In this methodology, a switch port is configured with a strict MAC address limit (e.g., a maximum of one secure MAC address per interface). If the switch detects frames originating from a MAC address that exceeds this threshold, a violation protocol is triggered, typically resulting in the immediate shutdown of the compromised port or the dropping of unauthorized frames. The operational logic of this count-based mechanism is illustrated in Fig. 4.

Gajo [4] evaluated the efficacy of Cisco’s Port Security feature to mitigate MAC flooding attacks using this approach. While this, successfully prevents CAM (Content Addressable Memory) table exhaustion at the hardware level, its real world application

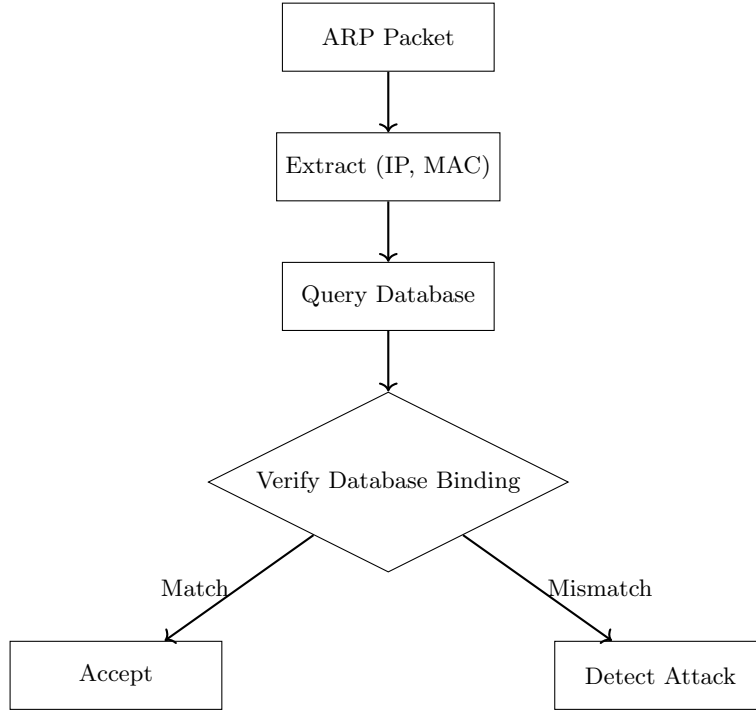


Fig. 3: Database-Based ARP Spoofing Detection Flowchart

presents significant operational challenges. As acknowledged by Gajo, strict MAC limiting is inflexible in modern enterprise environments that utilize VoIP phones with PC pass throughs or virtualized host environments requiring multiple MACs per physical port. Furthermore, utilizing a shutdown violation mode inherently introduces a self-inflicted Denial of Service (DoS) vulnerability. An attacker can intentionally generate a single spoofed frame to force the switch into an err-disable state, thereby disconnecting legitimate users. Finally, this technique only restricts MAC quantity, offering no protection against targeted MAC spoofing or cloning, where an attacker impersonates the single authorized device. The pros and cons of this approach are mentioned in Table 4.

Pros	Cons
High detection accuracy	High network overhead (extra broadcasts)
No protocol modification	Increased latency
Simple implementation	Reactive detection approach
Highly Scalable	

Table 4: Pros and Cons of Port Security Mechanism

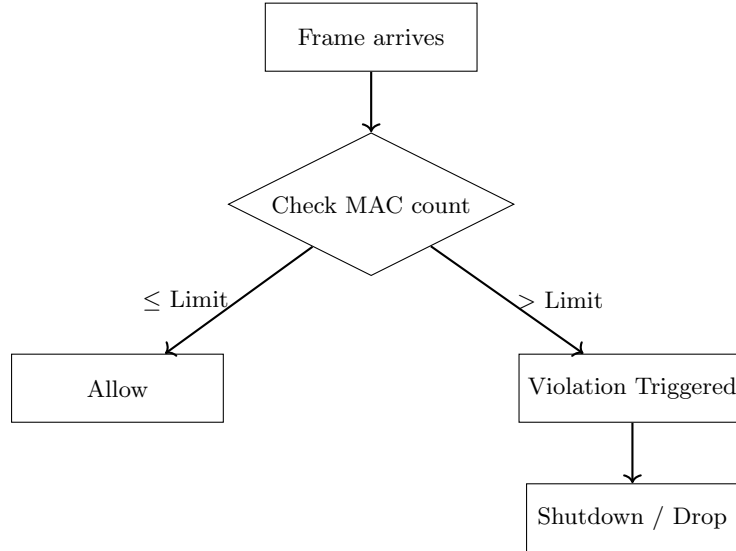


Fig. 4: MAC Count-Based Port Security Mechanism

2.5 Comparison of the discussed approaches

Despite the wide range of proposed solutions, several limitations remain unaddressed, as summarized in Table 5. Existing approaches either introduce significant overhead, lack real time enforcement, or depend on external components such as controllers, databases, or trained models. Furthermore, most techniques operate at higher layers of the network stack, making them unsuitable for high speed environments where rapid packet processing is critical.

Additionally, current solutions tend to focus on specific attack vectors, such as ARP spoofing or MAC flooding, rather than providing a unified and efficient mechanism for Layer-2 security. This fragmentation highlights the absence of a lightweight, scalable and high-performance solution capable of mitigating multiple Layer-2 attacks simultaneously.

To address these limitations, there is a need for a solution that provides:

- Early packet inspection (before kernel processing)
- Low latency and minimal overhead
- Real-time enforcement at line rate
- Flexibility to handle multiple Layer-2 attacks

In this context, eXpress Data Path (XDP) offers a promising approach by enabling packet processing at the driver level, allowing malicious packets to be detected and dropped at the earliest stage of the networking stack.

Approach	Accuracy	Scalability	Processing Overhead	Latency
ARP Broadcast Verification	Detects spoofed packets by comparing real time MAC responses with received ARP packets, resulting in high false negatives	Limited scalability due to frequent network wide ARP broadcasts in larger networks	Introduces moderate overhead due to continuous packet capture and broadcast based validation	Adds moderate delay because of additional ARP request response cycles
Machine Learning-Based Detection (SVM, MLP)	Achieves high detection accuracy by learning complex traffic patterns and identifying anomalies in real time	Highly scalable as models can adapt to large and dynamic network environments	Incurs high computational overhead due to training, feature extraction, and model inference	Moderate latency depending on model complexity and real-time processing requirements
Database-Driven IP MAC Binding Verification	Provides reliable detection by validating IP MAC mappings using a centralized database and hash verification	Moderately scalable but depends on database size and synchronization across network devices	Requires additional storage and lookup operations, leading to moderate processing overhead	Introduces moderate latency due to database queries and validation steps
Port Security (MAC Address Limiting)	Effectively prevents MAC flooding by restricting the number of MAC addresses per port but does not detect sophisticated spoofing attacks	Highly scalable as it is implemented directly on switches without centralized coordination 9	Very low overhead since enforcement is handled in hardware at the switch level	Minimal latency as decisions are made instantly at the data link layer

Table 5: Comparison of Existing Layer-2 Attack Mitigation Techniques

3 Design of Native XDP-Based Layer-2 Security Mechanism

3.1 Design Overview

The proposed system addresses Layer-2 attacks by combining a trusted binding source with line rate dataplane enforcement. Specifically, it leverages passive DHCP snooping to learn IP-MAC bindings and the eXpress Data Path (XDP) to enforce validation at the earliest stage of packet processing. A key design principle of this architecture is the strict separation of concerns between the control and data planes. The control plane, operating as a user space daemon, is responsible for asynchronously learning and maintaining state bindings derived from DHCP events. Conversely, the data plane utilizes XDP and Extended Berkeley Packet Filter (eBPF) programs to perform deterministic, per-packet security enforcement directly at the network interface card (NIC) driver level. This hybrid approach ensures that complex packet parsing and state management operations do not burden the fast path, allowing security decisions to be enforced with minimal latency and high throughput, as shown in Fig 5.

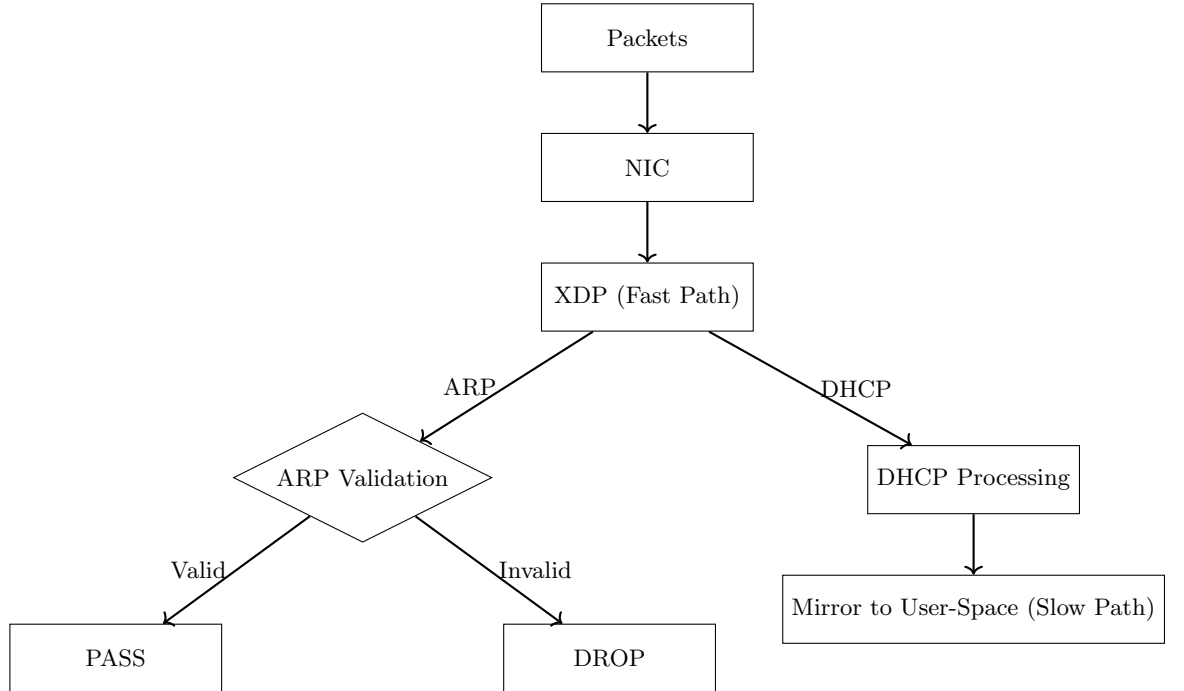


Fig. 5: XDP-Based Fast Path and Slow Path Packet Processing

3.2 Dynamic Binding Generation via DHCP Snooping

Accurate ARP validation intrinsically requires a reliable and dynamic mapping between IP addresses and MAC addresses. Instead of relying on static configurations or high latency external databases, the proposed system constructs this mapping dynamically by passively observing DHCP transactions. Because DHCP is responsible for assigning IP addresses to clients on a network, it serves as a highly reliable, trusted source of ground truth for IP MAC associations. However, DHCP packets are structurally complex, often containing variable-length Type Length Value (TLV) options that make them computationally expensive and unsuitable for full parsing within the strict bounds of XDP. Consequently, the system utilizes a hybrid ingestion mechanism..

The dynamic binding generation process is formalized below in Algorithm 1. When a packet arrives at the network interface, the XDP program performs a preliminary inspection. If the packet is identified as a DHCP transaction, the system utilizes a zero copy mirroring technique via an eBPF ring buffer. This mechanism allows the packet to continue along its normal forwarding path uninterrupted, while a mirrored copy is asynchronously pushed to the user space control plane as shown in Fig 6.

Algorithm 1 DHCP-Based Binding Map Update

Require: Packet P

```
1: if  $P$  is DHCP then
2:   Send  $P$  to user space via ring buffer

3:   In user space:
4:   if DHCP ACK received then
5:     Extract IP  $\leftarrow$  yiaddr
6:     Extract MAC  $\leftarrow$  chaddr
7:     binding_map[IP]  $\leftarrow$  MAC
8:     Record timestamp
9:   end if
10: end if
```

This hybrid design is critical for maintaining line rate processing. By decoupling the complex parsing of DHCP options from the fast path, the system avoids the strict processing constraints imposed by the eBPF in-kernel verifier. Furthermore, because the user space daemon performs map updates asynchronously, the creation of IP MAC bindings does not block or delay active packet forwarding. The use of eBPF maps guarantees atomic operations, ensuring that the state remains consistent between the control plane and the data plane even under high throughput conditions.

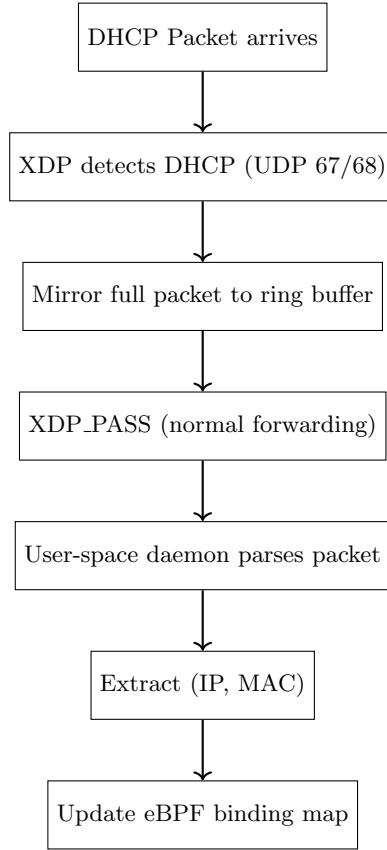


Fig. 6: DHCP Packet Handling using XDP and eBPF

3.3 Real-Time ARP Validation in XDP

Once the reliable IP MAC bindings are established in the eBPF maps, incoming ARP packets are subjected to strict validation within the XDP fast path to prevent spoofing attempts. The core enforcement logic extracts the source IP and MAC addresses from the ARP payload. It queries the eBPF binding map for the claimed IP address. If a mismatch occurs between the mapped MAC and the source MAC, the packet is immediately dropped before traversing the host's networking stack. This flow is illustrated in Fig 7. While a general ARP Validation Logic would follow an algorithm like Algorithm 2, wherein if the binding is not present in the eBPF map, we apply a controlled acceptance policy. However, gratuitous ARP packets have to be handled in a separate manner as they are heavily used in MITM attacks. So, if the source and target IPs match, the system strictly enforces the binding map check as shown in Algorithm 3.

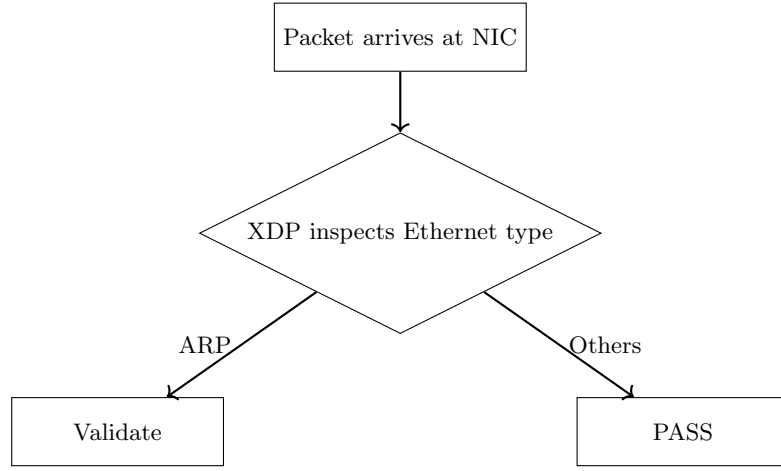


Fig. 7: XDP Ethernet Type Inspection and Handling

Algorithm 2 ARP Packet Validation using Binding Map

Require: ARP packet A

- 1: Extract IP_{src}, MAC_{src}
 - 2: $binding \leftarrow binding_map[IP_{src}]$
 - 3: **if** binding exists **then**
 - 4: **if** $MAC_{src} = binding$ **then**
 - 5: PASS
 - 6: **else**
 - 7: DROP
 - 8: **end if**
 - 9: **else**
 - 10: Apply controlled acceptance policy
 - 11: **end if**
-

Algorithm 3 Gratuitous ARP Handling with Strict Validation

Require: ARP packet A

- 1: Extract IP_{src}, IP_{target}
- 2: **if** $IP_{src} = IP_{target}$ **then**
- 3: $binding \leftarrow binding_map[IP_{src}]$
- 4: **if** binding exists **then**
- 5: **PASS**
- 6: **else**
- 7: **DROP**
- 8: **end if**
- 9: **end if**

3.4 Handling Initial State and Update Gap

In real world network environments, a race condition exists during initial device connection. An endpoint may transmit an initial ARP packet immediately upon connecting, before its DHCP transaction completes as shown in Fig 8. Furthermore, there is also a microsecond scale latency gap between the observation of a DHCP packet and the subsequent update of the binding map by the user-space daemon. This temporary state creates an operational vulnerability where the system cannot immediately verify legitimate early traffic. To mitigate this without blindly allowing arbitrary packets or excessively dropping legitimate connection requests, the system implements a controlled, rate limited acceptance strategy, as seen in Fig 9.

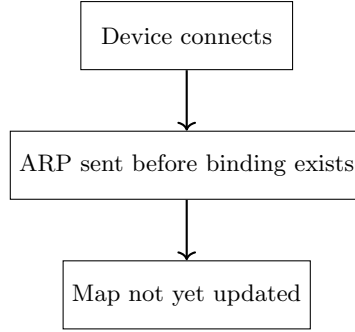


Fig. 8: Initial Connection Scenario Before Binding Map Update

Formalized in Algorithm 4 this mechanism allows a strict, minimal threshold of unverified ARP packets to pass, enabling necessary early communication and bridging the DHCP update gap. However, if the volume of unknown ARP requests exceeds the expected behavior of a normal host joining a network, the system assumes an active flood or spoofing attempt and begins dropping the packets. Once a DHCP binding is formally established, this fallback behavior is bypassed in favor of strict validation.

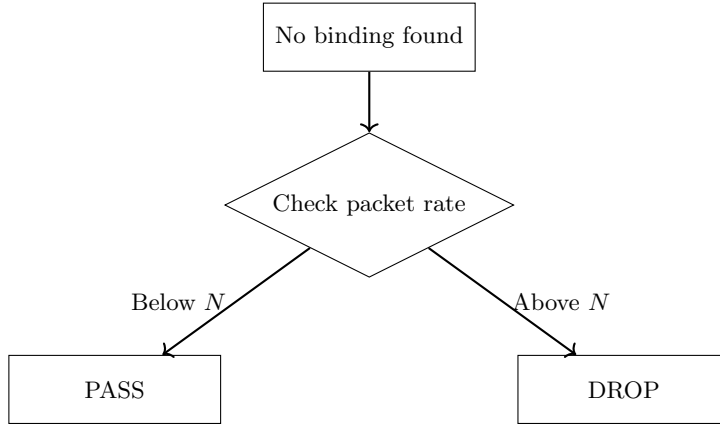


Fig. 9: Rate-Limiting Policy for Unknown Bindings

Algorithm 4 Rate Limiting for Unknown ARP Bindings

Require: ARP packet with unknown IP

```

1: binding  $\leftarrow$  binding_map[IP]
2: if binding not found then
3:   counter  $\leftarrow$  counter + 1
4:   if counter < threshold then
5:     PASS
6:   else
7:     DROP
8:   end if
9: end if

```

3.5 MAC Flooding Detection and Mitigation

To mitigate MAC flooding attacks where attackers attempt to exhaust switch forwarding tables via high volume of spoofed MAC addresses, the system employs a dynamic stateful tracking mechanism.

Upon packet ingress, the Ethernet header is parsed to extract the source MAC address, which is subsequently evaluated against a trusted binding map. Verified addresses are immediately forwarded. Unverified MAC addresses trigger the detection mechanism and are inserted into a tracking map. To guarantee structural resilience and prevent memory exhaustion during volumetric attacks, this tracking map enforces a Least Recently Used (LRU) eviction policy.

Following insertion, the system calculates the current volume of unique, unverified MAC addresses against a predefined threshold. This threshold serves as the primary anomaly indicator. If the aggregate count exceeds the threshold, the packet is classified

as malicious and discarded. Conversely, if the count remains within nominal limits, the packet is forwarded. This flow is illustrated in Fig 10 and formalised in Algorithm 5. Integrated with standard ARP validation, this mechanism provides comprehensive Layer-2 security.

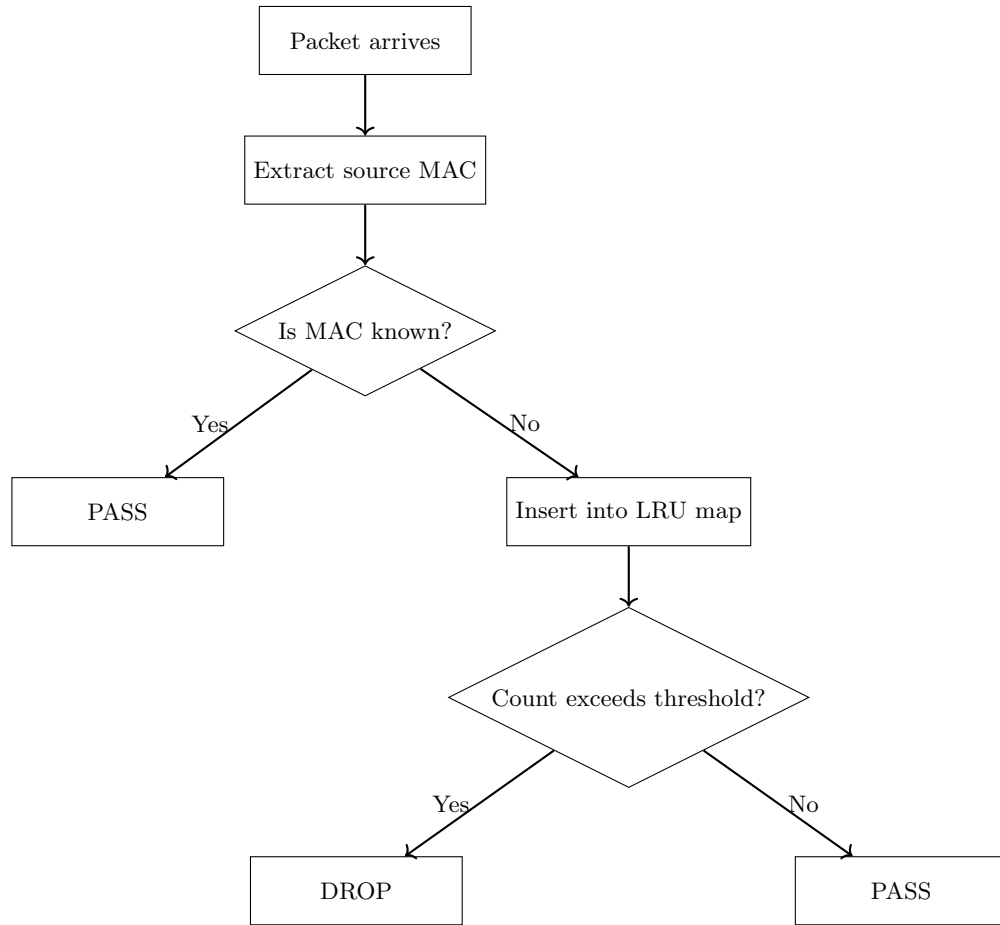


Fig. 10: MAC Learning with LRU and Threshold Based Control

Algorithm 5 MAC Tracking and Threshold Based Packet Filtering

Require: Packet P

```
1: Extract  $MAC_{src}$ 
2: if  $MAC_{src}$  not in binding_map then
3:   Insert  $MAC_{src}$  into mac_tracking_map
4:   if unique MAC count > threshold then
5:     DROP packet
6:   else
7:     PASS packet
8:   end if
9: else
10:  PASS packet
11: end if
```

3.6 Unified System Workflow

The independent logics for dynamic binding generation, ARP validation, and MAC anomaly tracking can be unified into a single Layer-2 security framework.

As detailed in Algorithm 6, ingress packets are initially subjected to a MAC flood mitigation heuristic. The source Ethernet address is evaluated against the `binding_map`. Unverified MACs are tracked using a Least Recently Used (LRU) map. If the volume of unique, unverified MACs on a given interface exceeds a predefined `FLOOD_THRESHOLD`, the packet is immediately discarded (`XDP_DROP`). This proactively prevents switch CAM table exhaustion prior to any deep payload inspection.

Packets bypassing the flood filter are subsequently segregated by EtherType. To facilitate dynamic state learning, DHCP traffic (UDP ports 67/68) is permitted (`XDP_PASS`) and concurrently mirrored to user-space via an eBPF ring buffer. The control plane asynchronously parses the DHCP ACK and inserts the learned IP-MAC binding into the kernel map. Standard IPv4/IPv6 payloads are routed unmodified, provided their source MACs comply with the established flooding thresholds.

The core enforcement logic, depicted in Fig. 11, applies rigorous inspection to ARP payloads. The XDP program extracts the sender IP, target IP, and sender MAC. If a binding exists, the packet is forwarded only if the payload MAC exactly matches the trusted state; mismatches are flagged as ARP poisoning and instantly dropped.

To address the microsecond-scale race condition between a client's initial ARP broadcast and the asynchronous DHCP map update, the system evaluates unmapped ARP requests. Standard ARP requests lacking an established binding are subjected to a rate limit, permitting initial network discovery. Conversely, unverified Gratuitous ARPs (GARPs), where the sender and target IPs are identical are strictly dropped to mitigate the primary vector for Man-in-the-Middle (MitM) cache poisoning.

This comprehensive processing pipeline is formalized in Algorithm 6 and illustrated in Fig. 11.

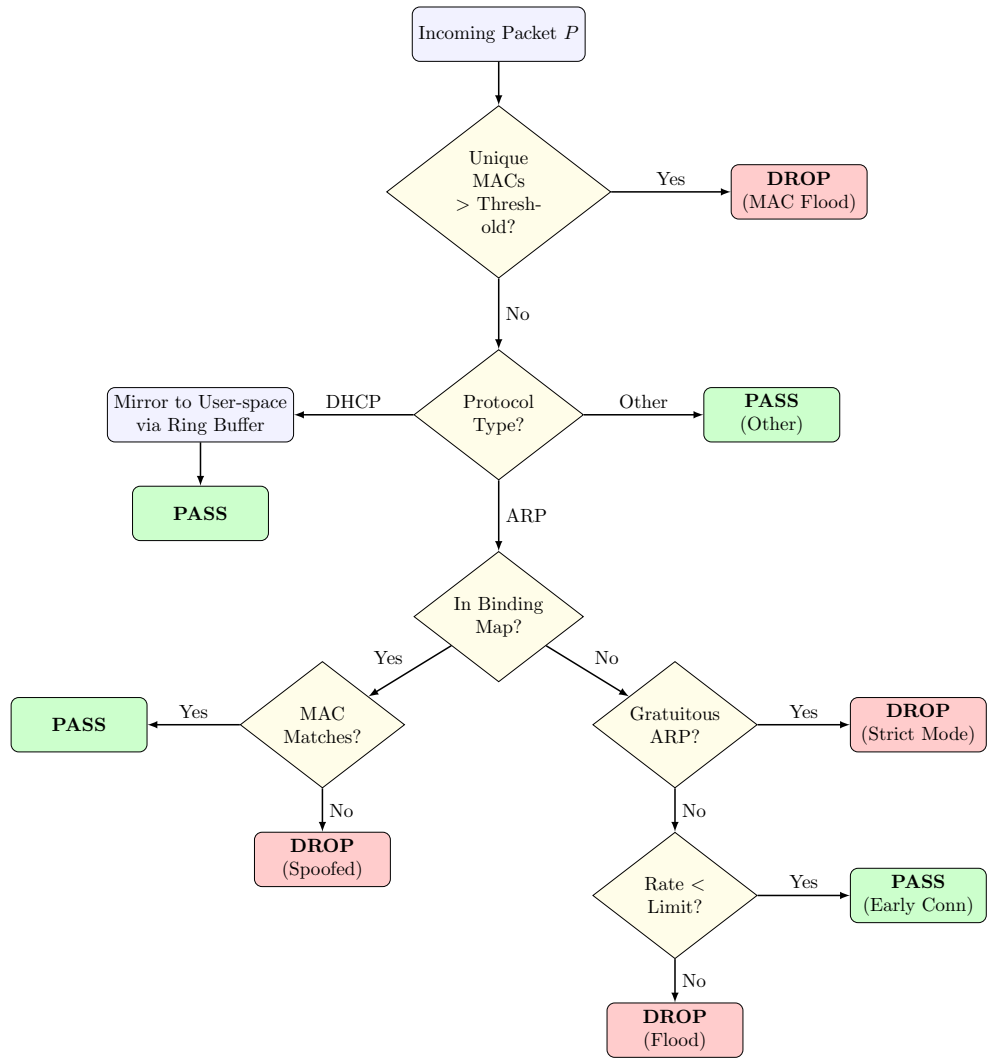


Fig. 11: Unified XDP-Based Layer-2 Security Enforcement Flowchart

Algorithm 6 Unified XDP-Based Layer-2 Security Enforcement

Require: Incoming Packet P at the XDP hook

```
1: Extract  $MAC_{src}$  and EtherType from Ethernet header

2: Phase 1: MAC Flooding Protection (Early Drop)
3: if  $MAC_{src} \notin \text{binding\_map}$  then
4:   Insert/Update  $MAC_{src}$  in mac_tracking_map (LRU)
5:   if Unique MAC count > FLOOD_THRESHOLD then
6:     return DROP
7:   end if
8: end if

9: Phase 2: Protocol Demultiplexing & Enforcement
10: if  $P$  is DHCP (UDP 67/68) then
11:   Mirror  $P$  to user-space daemon via eBPF ring buffer
12:   return PASS
13: else if  $P$  is ARP then
14:   Extract  $IP_{src}$ ,  $IP_{target}$ , and  $MAC_{payload}$ 
15:   binding  $\leftarrow$  binding_map[ $IP_{src}$ ]

16:   if binding exists then
17:     if  $MAC_{payload} = \text{binding}$  then
18:       return PASS
19:     else
20:       return DROP
21:     end if
22:   else
23:     if  $IP_{src} = IP_{target}$  then
24:       return DROP
25:     else
26:       if Packet rate for  $IP_{src} < \text{RATE\_LIMIT}$  then
27:         return PASS
28:       else
29:         return DROP
30:       end if
31:     end if
32:   end if
33: else
34:   return PASS
35: end if
```

4 Implementation

The above represented Algorithms have been implemented in C programming language in the subsequent sub sections.

4.1 eBPF Header File

The header file `xdp_l2_security.h` acts as the interface between the eBPF kernel space program and the user space daemon, ensuring that both components share a unified view of the network state. It defines the core data structures for IP-MAC binding, such as the `mac_addr` and `dhcp_packet` structures, which are used to parse DHCP messages and populate stateful security maps. Furthermore, it establishes the fundamental security logic by setting tunable thresholds specifically `FLOOD_THRESHOLD` for MAC flooding detection and `RATE_LIMIT` for ARP rate limiting—allowing the system to distinguish between legitimate network discovery and active attacker traffic.

Listing 1: `xdp_l2_security.h`: Shared definitions for kernel and user-space

```
#ifndef XDP_L2_SECURITY_H
#define XDP_L2_SECURITY_H

#ifndef FLOOD_THRESHOLD
#define FLOOD_THRESHOLD 100
#endif

#ifndef RATE_LIMIT
#define RATE_LIMIT 5
#endif

#define RINGBUF_SIZE (256 * 1024)
#define MAX_BINDINGS 1024
#define MAX_MAC_ENTRIES 4096
#define MAX_RATE_ENTRIES 1024

enum stats_index {
    STATS_TOTAL_PACKETS = 0,
    STATS_PASSED,
    STATS_DROPPED_ARP_SPOOF,
    STATS_DROPPED_GARP,
    STATS_DROPPED_MAC_FLOOD,
    STATS_DROPPED_RATE_LIMIT,
    STATS_DHCP_MIRRORED,
    STATS_ARP_PASSED,
    STATS_MAX
};

#ifndef HAVE_VMLINUX_H
struct mac_addr {
    unsigned char addr[6];
} __attribute__((packed));
```

```

#endif

#define MAX_DHCP_PACKET_SIZE 1500

struct dhcp_event {
    unsigned int pkt_len;
    unsigned char pkt_data[MAX_DHCP_PACKET_SIZE];
};

#define DHCP_SERVER_PORT 67
#define DHCP_CLIENT_PORT 68

#define DHCP_MSG_DISCOVER 1
#define DHCP_MSG_OFFER 2
#define DHCP_MSG_REQUEST 3
#define DHCP_MSG_DECLINE 4
#define DHCP_MSG_ACK 5
#define DHCP_MSG_NAK 6
#define DHCP_MSG_RELEASE 7
#define DHCP_MSG_INFORM 8

#define DHCP_OPT_MSG_TYPE 53
#define DHCP_OPT_END 255
#define DHCP_OPT_PAD 0

#define DHCP_MAGIC_COOKIE 0x63825363

struct dhcp_packet {
    unsigned char op;
    unsigned char htype;
    unsigned char hlen;
    unsigned char hops;
    unsigned int xid;
    unsigned short secs;
    unsigned short flags;
    unsigned int ciaddr;
    unsigned int yiaddr;
    unsigned int siaddr;
    unsigned int giaddr;
    unsigned char chaddr[16];
    unsigned char sname[64];
    unsigned char file[128];
    unsigned int magic;
} __attribute__((packed));

#endif

```

4.2 User Space Daemon Code

The user-space control plane, implemented in `l2_security_daemon.c`, serves as the orchestration layer for the XDP security framework. Its primary responsibility is the asynchronous management of the `binding_map` through DHCP snooping, as described in Algorithm 1. By polling the eBPF ring buffer, the daemon intercepts mirrored DHCP ACK and RELEASE messages to dynamically update the trusted IP-MAC associations without interfering with the data plane's line-rate performance. It also provides an option for manual binding entry overrides. This hybrid architecture ensures that complex protocol parsing is offloaded to user space, maintaining a minimalist and high performance kernel fast path.

Listing 2: `l2_security_daemon.c`: User-space control plane and DHCP snooping logic

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <signal.h>
#include <unistd.h>
#include <time.h>
#include <arpa/inet.h>
#include <net/if.h>
#include <linux/if_ether.h>
#include <linux/if_link.h>
#include <bpf/libbpf.h>
#include <bpf/bpf.h>
#include "xdp_l2_security.h"
#include "xdp_l2_security.skel.h"

static volatile int running = 1;
static int verbose = 0;

#define MAX_IFACES 8
#define MAX_STATIC_BINDINGS 16

static const char *stat_names[] = {
    [STATS_TOTAL_PACKETS] = "Total Packets",
    [STATS_PASSED] = "Packets Passed",
    [STATS_DROPPED_ARP_SPOOF] = "ARP Spoof Drops",
    [STATS_DROPPED_GARP] = "Gratuitous ARP Drops",
    [STATS_DROPPED_MAC_FLOOD] = "MAC Flood Drops",
    [STATS_DROPPED_RATE_LIMIT] = "Rate Limit Drops",
    [STATS_DHCP_MIRRORED] = "DHCP Packets Mirrored",
    [STATS_ARP_PASSED] = "ARP Packets Passed",
};

static void sig_handler(int sig)
{
    (void)sig;
```

```

    running = 0;
}

static void mac_to_str(const unsigned char *mac, char *buf,
    size_t buf_sz)
{
    snprintf(buf, buf_sz, "%02x:%02x:%02x:%02x:%02x:%02x",
        mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);
}

static int parse_mac(const char *str, unsigned char *mac)
{
    int vals[6];
    if (sscanf(str, "%x:%x:%x:%x:%x:%x",
        &vals[0], &vals[1], &vals[2],
        &vals[3], &vals[4], &vals[5]) != 6)
        return -1;
    for (int i = 0; i < 6; i++)
        mac[i] = (unsigned char)vals[i];
    return 0;
}

static void time_str(char *buf, size_t sz)
{
    time_t now = time(NULL);
    struct tm *tm = localtime(&now);
    strftime(buf, sz, "%Y-%m-%d %H:%M:%S", tm);
}

static int binding_map_fd = -1;

static int parse_dhcp_msg_type(const unsigned char *options, int
    options_len)
{
    int i = 0;
    while (i < options_len) {
        unsigned char opt_code = options[i];
        if (opt_code == DHCP_OPT_PAD) {
            i++;
            continue;
        }
        if (opt_code == DHCP_OPT_END)
            break;
        if (i + 1 >= options_len)
            break;
        unsigned char opt_len = options[i + 1];
        if (opt_code == DHCP_OPT_MSG_TYPE) {
            if (opt_len >= 1 && i + 2 < options_len)
                return options[i + 2];
        }
    }
}

```

```

        i += 2 + opt_len;
    }
    return -1;
}

static int handle_dhcp_event(void *ctx, void *data, size_t
data_sz)
{
    (void)ctx;
    struct dhcp_event *evt = data;
    if (data_sz < sizeof(struct dhcp_event))
        return 0;
    unsigned int pkt_len = evt->pkt_len;
    unsigned char *pkt = evt->pkt_data;
    if (pkt_len < 14 + 20 + 8 + 236 + 4)
        return 0;
    unsigned char *ip_hdr = pkt + 14;
    int ip_hdr_len = (ip_hdr[0] & 0x0f) * 4;
    unsigned char *dhcp_start = ip_hdr + ip_hdr_len + 8;
    int dhcp_offset = (dhcp_start - pkt);
    int dhcp_len = pkt_len - dhcp_offset;
    if (dhcp_len < 240)
        return 0;
    struct dhcp_packet *dhcp = (struct dhcp_packet *)dhcp_start;
    if (ntohl(dhcp->magic) != DHCP_MAGIC_COOKIE)
        return 0;
    unsigned char *options = dhcp_start + 240;
    int options_len = dhcp_len - 240;
    int msg_type = parse_dhcp_msg_type(options, options_len);
    char ts[32];
    time_str(ts, sizeof(ts));
    if (msg_type == DHCP_MSG_ACK) {
        __be32 assigned_ip = dhcp->yiaddr;
        struct mac_addr client_mac;
        memcpy(client_mac.addr, dhcp->chaddr, 6);
        int err = bpf_map_update_elem(binding_map_fd, &
            assigned_ip,
                                &client_mac, BPF_ANY);

        char ip_str[INET_ADDRSTRLEN];
        char mac_str[18];
        inet_ntop(AF_INET, &assigned_ip, ip_str, sizeof(ip_str));
        mac_to_str(client_mac.addr, mac_str, sizeof(mac_str));
        if (err == 0) {
            printf("[%s] BINDING ADDED: %s -> %s\n", ts, ip_str,
                mac_str);
        }
    }
    else if (msg_type == DHCP_MSG_RELEASE) {
        __be32 client_ip = dhcp->ciaddr;
        char ip_str[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &client_ip, ip_str, sizeof(ip_str));
    }
}

```



```

        bpf_map_delete_elem(binding_map_fd, &client_ip);
        printf("[%s] BINDING REMOVED: %s\n", ts, ip_str);
    }
    return 0;
}

static void list_bindings(int map_fd)
{
    __be32 key = 0, next_key;
    struct mac_addr value;
    int count = 0;
    while (bpf_map_get_next_key(map_fd, &key, &next_key) == 0) {
        if (bpf_map_lookup_elem(map_fd, &next_key, &value) == 0)
        {
            char ip_str[INET_ADDRSTRLEN];
            char mac_str[18];
            inet_ntop(AF_INET, &next_key, ip_str, sizeof(ip_str))
                ;
            mac_to_str(value.addr, mac_str, sizeof(mac_str));
            printf("%s -> %s\n", ip_str, mac_str);
            count++;
        }
        key = next_key;
    }
    printf("Total bindings: %d\n", count);
}

static void show_stats(int map_fd)
{
    int nr_cpus = libbpf_num_possible_cpus();
    if (nr_cpus < 0) return;
    __u64 values[nr_cpus];
    for (__u32 i = 0; i < STATS_MAX; i++) {
        if (bpf_map_lookup_elem(map_fd, &i, values) != 0)
            continue;
        __u64 total = 0;
        for (int cpu = 0; cpu < nr_cpus; cpu++)
            total += values[cpu];
        printf("%s: %llu\n", stat_names[i], total);
    }
}

int main(int argc, char **argv)
{
    const char *ifaces[MAX_IFACES];
    int n_ifaces = 0;
    int mode_list = 0, mode_stats = 0;
    const char *add_ip = NULL, *add_mac = NULL, *del_ip = NULL;
    const char *static_ips[MAX_STATIC_BINDINGS];
    const char *static_macs[MAX_STATIC_BINDINGS];

```

```

int n_static = 0;
for (int i = 1; i < argc; i++) {
    if (strcmp(argv[i], "--iface") == 0 && i + 1 < argc) {
        if (n_ifaces < MAX_IFACES) ifaces[n_ifaces++] = argv
            [i+1];
    } else if (strcmp(argv[i], "--list-bindings") == 0)
        mode_list = 1;
    else if (strcmp(argv[i], "--stats") == 0) mode_stats = 1;
    else if (strcmp(argv[i], "--add-binding") == 0 && i + 2 <
        argc) {
        add_ip = argv[i+1]; add_mac = argv[i+2];
    } else if (strcmp(argv[i], "--del-binding") == 0 && i + 1
        < argc) {
        del_ip = argv[i+1];
    } else if (strcmp(argv[i], "--static-binding") == 0 && i
        + 2 < argc) {
        if (n_static < MAX_STATIC_BINDINGS) {
            static_ips[n_static] = argv[i+1];
            static_macs[n_static] = argv[i+2];
            n_static++;
        }
    } else if (strcmp(argv[i], "--verbose") == 0) verbose =
        1;
}
struct xdp_l2_security_bpf *skel = xdp_l2_security_bpf__open
    ();
if (!skel) return 1;
int err = xdp_l2_security_bpf__load(skel);
if (err) { xdp_l2_security_bpf__destroy(skel); return 1; }
binding_map_fd = bpf_map__fd(skel->maps.binding_map);
int stats_map_fd = bpf_map__fd(skel->maps.stats_map);
if (mode_list) { list_bindings(binding_map_fd);
    xdp_l2_security_bpf__destroy(skel); return 0; }
if (mode_stats) { show_stats(stats_map_fd);
    xdp_l2_security_bpf__destroy(skel); return 0; }
for (int i = 0; i < n_static; i++) {
    struct in_addr addr;
    inet_pton(AF_INET, static_ips[i], &addr);
    struct mac_addr mac;
    parse_mac(static_macs[i], mac.addr);
    bpf_map_update_elem(binding_map_fd, &addr.s_addr, &mac,
        BPF_ANY);
}
int prog_fd = bpf_program__fd(skel->progs.xdp_l2_security);
for (int i = 0; i < n_ifaces; i++) {
    unsigned int ifindex = if_nametoindex(ifaces[i]);
    LIBBPF_OPTS(bpf_xdp_attach_opts, attach_opts);
    err = bpf_xdp_attach(ifindex, prog_fd, XDP_FLAGS_DRV_MODE
        , &attach_opts);
}

```

```

        if (err) bpf_xdp_attach(ifindex, prog_fd,
                                XDP_FLAGS_SKB_MODE, &attach_opts);
    }
    struct ring_buffer *rb = ring_buffer__new(bpf_map__fd(skel->
        maps.events_rb), handle_dhcp_event, NULL, NULL);
    signal(SIGINT, sig_handler);
    signal(SIGTERM, sig_handler);
    while (running) {
        err = ring_buffer__poll(rb, 100);
        if (err < 0 && err != -EINTR) break;
    }
    show_stats(stats_map_fd);
    for (int i = 0; i < n_ifaces; i++) {
        unsigned int ifindex = if_nametoindex(ifaces[i]);
        if (ifindex > 0) {
            LIBBPF_OPTS(bpf_xdp_attach_opts, detach_opts);
            bpf_xdp_detach(ifindex, 0, &detach_opts);
        }
    }
    ring_buffer__free(rb);
    xdp_l2_security_bpf__destroy(skel);
    return 0;
}

```

4.3 eBPF Kernel Space Code

The kernel-space enforcement logic, implemented in `xdp_l2_security.bpf.c`, provides a unified high-performance data plane for mitigating Layer-2 threats as described in Algorithm 6. Operating at the earliest possible point in the Linux networking stack, the program executes a multi phase validation pipeline: Phase 1 implements MAC flooding protection (Algorithm 5) by utilizing an LRU hash map to track unverified source MAC addresses and dropping traffic that exceeds the `FLOOD_THRESHOLD`. Phase 2 performs protocol demultiplexing, where DHCP traffic is mirrored to user-space via a zero copy ring buffer (Algorithm 1), while ARP traffic undergoes strict stateful validation (Algorithms 2–4) against the `binding_map`. By leveraging eBPF maps for $O(1)$ lookups and atomic statistics gathering, the implementation ensures that security policies such as dropping spoofed ARP replies or rate limiting unknown sources are enforced with minimal latency and negligible CPU overhead.

Listing 3: `xdp_l2_security.bpf.c`: Unified Kernel-Space Security Enforcement

```

#include "vmlinux.h"
#define HAVE_VMLINUX_H 1
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>
#include "xdp_l2_security.h"

#define ETH_P_ARP 0x0806
#define ETH_P_IP 0x0800

```

```

#define ETH_ALEN 6
#define IPPROTO_UDP 17
#define ARP_OP_REQUEST 1
#define ARP_OP_REPLY 2

struct arp_hdr {
    __be16 ar_hrd;
    __be16 ar_pro;
    __u8 ar_hln;
    __u8 ar_pln;
    __be16 ar_op;
    __u8 ar_sha[6];
    __be32 ar_sip;
    __u8 ar_tha[6];
    __be32 ar_tip;
} __attribute__((packed));

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, MAX_BINDINGS);
    __type(key, __be32);
    __type(value, struct mac_addr);
} binding_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __uint(max_entries, MAX_MAC_ENTRIES);
    __type(key, struct mac_addr);
    __type(value, __u64);
} mac_tracking_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 1);
    __type(key, __u32);
    __type(value, __u64);
} mac_count_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_LRU_HASH);
    __uint(max_entries, MAX_RATE_ENTRIES);
    __type(key, __be32);
    __type(value, __u64);
} rate_limit_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(max_entries, STATS_MAX);
    __type(key, __u32);
    __type(value, __u64);
}

```

```

} stats_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, RINGBUF_SIZE);
} events_rlb SEC(".maps");

static __always_inline void stats_inc(__u32 idx)
{
    __u64 *val = bpf_map_lookup_elem(&stats_map, &idx);
    if (val)
        __sync_fetch_and_add(val, 1);
}

static __always_inline int mac_equal(const unsigned char *a,
    const unsigned char *b)
{
    return (a[0] == b[0]) && (a[1] == b[1]) && (a[2] == b[2]) &&
        (a[3] == b[3]) && (a[4] == b[4]) && (a[5] == b[5]);
}

static __always_inline int is_broadcast_mac(const unsigned char *
    mac)
{
    return (mac[0] & mac[1] & mac[2] & mac[3] & mac[4] & mac[5])
        == 0xff;
}

static __always_inline int is_zero_mac(const unsigned char *mac)
{
    return (mac[0] | mac[1] | mac[2] | mac[3] | mac[4] | mac[5])
        == 0;
}

SEC(".xdp")
int xdp_l2_security(struct xdp_md *ctx)
{
    void *data      = (void *) (long) ctx->data;
    void *data_end  = (void *) (long) ctx->data_end;

    stats_inc(STATS_TOTAL_PACKETS);

    struct ethhdr *eth = data;
    if ((void *) (eth + 1) > data_end)
        return XDP_PASS;

    __u16 eth_type = bpf_ntohs(eth->h_proto);
    struct mac_addr src_mac;
    __builtin_memcpy(src_mac.addr, eth->h_source, ETH_ALEN);

```

```

if (!is_broadcast_mac(src_mac.addr) && !is_zero_mac(src_mac.
addr)) {
    __u64 *existing = bpf_map_lookup_elem(&mac_tracking_map,
&src_mac);
    if (!existing) {
        __u64 ts = bpf_ktime_get_ns();
        bpf_map_update_elem(&mac_tracking_map, &src_mac, &ts,
BPF_ANY);

        __u32 zero = 0;
        __u64 *cnt = bpf_map_lookup_elem(&mac_count_map, &
zero);
        if (cnt) {
            __u64 new_count = __sync_add_and_fetch(cnt, 1);
            if (new_count > FLOOD_THRESHOLD) {
                stats_inc(STATS_DROPPED_MAC_FLOOD);
                return XDP_DROP;
            }
        }
    }
}

if (eth_type == ETH_P_IP) {
    struct iphdr *iph = (void *)(eth + 1);
    if ((void *)(iph + 1) > data_end)
        return XDP_PASS;

    if (iph->protocol == IPPROTO_UDP) {
        struct udphdr *udph = (void *)iph + (iph->ihl * 4);
        if ((void *)(udph + 1) > data_end)
            return XDP_PASS;

        __u16 src_port = bpf_ntohs(udph->source);
        __u16 dst_port = bpf_ntohs(udph->dest);

        if (src_port == DHCP_SERVER_PORT || src_port ==
DHCP_CLIENT_PORT ||
            dst_port == DHCP_SERVER_PORT || dst_port ==
DHCP_CLIENT_PORT) {

            __u32 pkt_len = data_end - data;
            if (pkt_len > MAX_DHCP_PACKET_SIZE)
                pkt_len = MAX_DHCP_PACKET_SIZE;

            struct dhcp_event *evt;
            evt = bpf_ringbuf_reserve(&events_rb, sizeof(
struct dhcp_event), 0);
            if (evt) {
                evt->pkt_len = pkt_len;
                if (pkt_len <= MAX_DHCP_PACKET_SIZE) {

```

```

        bpf_probe_read_kernel(evt->pkt_data,
                               pkt_len, data);
    }
    bpf_ringbuf_submit(evt, 0);
    stats_inc(STATS_DHCP_MIRRORED);
}
stats_inc(STATS_PASSED);
return XDP_PASS;
}
}
stats_inc(STATS_PASSED);
return XDP_PASS;
}

if (eth_type == ETH_P_ARP) {
    struct arp_hdr *arph = (void *) (eth + 1);
    if ((void *) (arph + 1) > data_end)
        return XDP_PASS;

    if (bpf_ntohs(arph->ar_hrd) != 1 || bpf_ntohs(arph->
ar_pro) != ETH_P_IP ||
        arph->ar_hln != ETH_ALEN || arph->ar_pln != 4)
    {
        stats_inc(STATS_PASSED);
        return XDP_PASS;
    }

    __be32 ip_src    = arph->ar_sip;
    __be32 ip_target = arph->ar_tip;

    struct mac_addr *binding = bpf_map_lookup_elem(&
binding_map, &ip_src);

    if (binding) {
        if (mac_equal(arph->ar_sha, binding->addr)) {
            stats_inc(STATS_ARP_PASSED);
            stats_inc(STATS_PASSED);
            return XDP_PASS;
        } else {
            stats_inc(STATS_DROPPED_ARP_SPOOF);
            return XDP_DROP;
        }
    }

    if (ip_src == ip_target) {
        stats_inc(STATS_DROPPED_GARP);
        return XDP_DROP;
    }
}

```

```

        __u64 *counter = bpf_map_lookup_elem(&rate_limit_map, &
        ip_src);
    if (counter) {
        __u64 cnt = __sync_add_and_fetch(counter, 1);
        if (cnt > RATE_LIMIT) {
            stats_inc(STATS_DROPPED_RATE_LIMIT);
            return XDP_DROP;
        }
    } else {
        __u64 init_val = 1;
        bpf_map_update_elem(&rate_limit_map, &ip_src, &
        init_val, BPF_ANY);
    }

    stats_inc(STATS_ARP_PASSED);
    stats_inc(STATS_PASSED);
    return XDP_PASS;
}

stats_inc(STATS_PASSED);
return XDP_PASS;
}

char _license[] SEC("license") = "GPL";

```

5 Results and Analysis

Subsection 5.1 explains the experiment environment in which the simulation of the above code was carried out. Upon multiple executions of the above given code, the result and its subsequent analysis is discussed in section 5.2

5.1 Experiment/Simulation Environment Detail

The proposed XDP based Layer-2 security framework was evaluated in an isolated virtual network environment deployed on a Proxmox VE 8.4.14 hypervisor. The testbed comprises four virtual machines interconnected via internal virtual bridges (vmb1, vmb2, vmb3) with no external network connectivity, ensuring complete isolation of the experimental traffic. All VMs utilize VirtIO (virtio-net) network interfaces, which support native XDP attachment at the driver level, providing a realistic analogue to bare-metal NIC behavior.

VM-B serves as the central Layer-2 switch running the XDP security daemon with three bridge interfaces (ens19, ens20, ens21). VM-A acts as the attacker, VM-C as the legitimate client, and VM-D as the DHCP server (ISC DHCP, subnet 192.168.100.0/24). The XDP program is compiled with Clang targeting BPF and loaded via libbpf with native (driver-mode) attachment. Attack simulation utilizes arpspoof (dsniff), macof, and custom Scapy scripts.

Component	Specification
Hypervisor	Proxmox VE 8.4.14
Host CPU	Intel i7 13650-HX
Host RAM	16GB DDR5
VM-A (Attacker)	4 vCPUs, 2GB RAM, Ubuntu 22.04 LTS
VM-B (Switch/XDP)	6 vCPUs, 4GB RAM, Ubuntu 22.04 LTS
VM-C (Client)	6 vCPUs, 4GB RAM, Ubuntu 22.04 LTS
VM-D (DHCP Server)	6 vCPUs, 4GB RAM, Ubuntu 22.04 LTS
NIC Type	VirtIO (<code>virtio_net</code>)
XDP Mode	Native (Driver Mode)
Kernel Version	6.8.0-107-generic
Compiler	Clang 18.1.3
libbpf Version	1.3.0
Network Topology	3 isolated bridges (<code>vmbr1/vmbr2/vmbr3</code>)
Subnet	192.168.100.0/24
FLOOD_THRESHOLD	100
RATE_LIMIT	5

Table 6: Experimental Environment Specifications

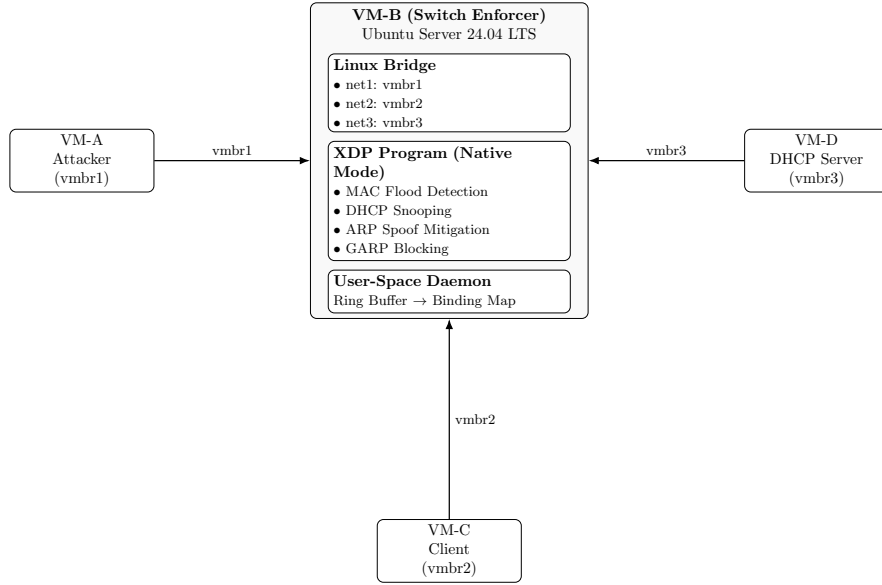


Fig. 12: Proxmox Host Architecture. VM-B acts as the security gateway, hosting the XDP kernel fast-path and the user-space control plane to protect the virtual network bridge.

5.2 Results with Detailed Analysis

This section presents a detailed evaluation of the proposed XDP based Layer 2 security framework across both security effectiveness and forwarding performance metrics. Subsection 5.2.1 analyzes security effectiveness by evaluating the system’s ability to detect and mitigate ARP spoofing, MAC flooding, and related attacks, along with false positive and false negative behavior. Subsection 5.2.2 examines forwarding performance in terms of throughput, measuring TCP/UDP bandwidth and packet processing rates under different workloads. Subsection 5.2.3 evaluates latency by analyzing round trip time under idle and loaded conditions to quantify the overhead of XDP based inspection. Subsection 5.2.4 investigates throughput resilience during active attack scenarios, highlighting the system’s ability to sustain legitimate traffic under adversarial conditions. Subsection ?? analyzes CPU utilization to assess computational overhead and scalability, while Subsection 5.2.6 evaluates the memory footprint of the framework to demonstrate its lightweight design and suitability for resource-constrained environments. All throughput and latency measurements are conducted using iperf3 (v3.16) between VM-C and VM-D through the XDP-protected bridge, with and without the XDP program attached.

5.2.1 Security Effectiveness: Attack Detection

The ARP spoofing detection achieves a 100% detection rate for all IP addresses with established DHCP bindings. When the attacker (VM-A, MAC bc:24:11:54:c1:6f) transmitted spoofed ARP replies claiming the IP of VM-C (192.168.100.100) or VM-D (192.168.100.1), the XDP program compared the ARP payload’s sender MAC (`ar_sha`) against the trusted binding in the eBPF hash map. Every mismatch resulted in an immediate `XDP_DROP` at the driver level, before the packet entered the kernel networking stack. Notably, gratuitous ARP packets where the sender IP equals the target IP were also caught by this binding validation, as the MAC mismatch check (Algorithm 2, lines 16–21 of Algorithm 6) executes before the dedicated GARP handler (Algorithm 3, lines 23–24).

The MAC flooding mitigation demonstrated a 99.99% drop rate, successfully blocking 1,754,503 out of ~1,754,600 attack packets. The `macof` tool generated Ethernet frames with randomized source MAC addresses at approximately 77,000 packets per second. The first 100 unique MACs were admitted during the learning window (governed by `FLOOD_THRESHOLD = 100`), after which every subsequent new MAC triggered an immediate drop. This threshold-based design, formalized in Algorithm 5, represents a deliberate tradeoff: setting the threshold too low risks false positives during normal network growth, while setting it too high allows more initial attack packets through. In practice, the 100-packet window represents a negligible fraction (0.006%) of the total attack volume.

Attack Type	Packets Sent	Packets Dropped	Rate	FN
ARP Spoofing	76,200	144,020*	100%	0
MAC Flooding	~1,754,600	1,754,503	99.99%	~100 [†]
Gratuitous ARP	67,800	(as spoof)**	100%	0
Combined Total	~1,898,600	1,898,523	99.99%	~100
Legitimate Traffic	216	0	N/A	N/A
False Positives			0%	

* ARP spoof drops include GARP packets caught by binding-MAC mismatch

** GARPs detected as spoofing due to MAC validation order

† First FLOOD_THRESHOLD packets pass during learning window

Table 7: Security Effectiveness — Detection Rate per Attack Vector

5.2.2 Forwarding Performance: Throughput

As shown in Table 8 and Fig. 13, the XDP program introduces negligible throughput overhead for legitimate traffic forwarding. Single-stream TCP throughput with XDP (11,349 Mbps) was comparable to the baseline (10,515 Mbps), with the slight variation attributed to normal fluctuation in the virtualized environment. UDP throughput remained virtually identical (3,795 vs. 3,775 Mbps), and small-packet processing sustained approximately 340,000 packets per second with only a 1.7% reduction. The multi-stream TCP test showed a larger overhead (24.9%), which is attributed to CPU contention between XDP processing and multiple parallel TCP flows in the VM rather than inherent XDP cost. These results confirm that the XDP fast-path architecture processes security logic at the driver level without meaningful degradation to forwarding performance.

Metric	Without XDP	With XDP	Overhead
TCP Throughput (1 stream)	10,515 Mbps	11,349 Mbps	-7.9%*
TCP Throughput (4 streams)	20,009 Mbps	15,021 Mbps	+24.9%**
UDP Throughput	3,775 Mbps	3,795 Mbps	-0.5%
UDP PPS (64-byte packets)	346K/s	340K/s	+1.7%

* Negative overhead indicates XDP performed better (within margin of error)

** Multi-stream overhead attributed to CPU contention in virtualized environment

Table 8: Forwarding Throughput — With and Without XDP Attached

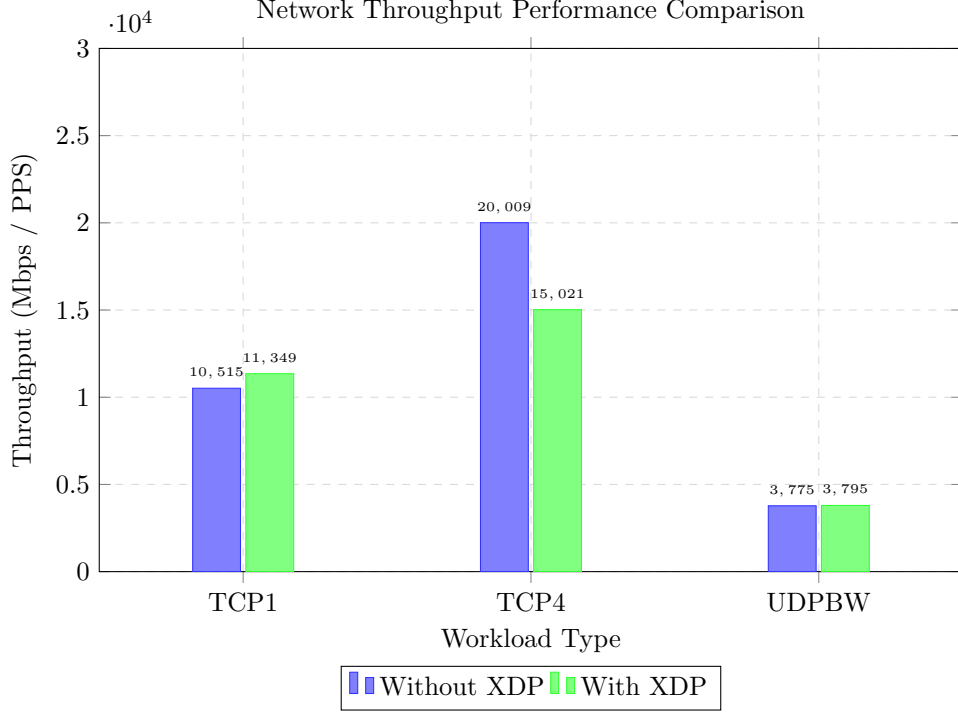


Fig. 13: Throughput Comparison (With and Without XDP). The XDP framework demonstrates negligible impact on baseline TCP performance while maintaining line-rate processing for high-frequency UDP packet streams.

5.2.3 Forwarding Performance: Latency

Latency measurements, as shown in Table 9 and Fig. 14, confirm that XDP-based packet inspection adds sub-millisecond overhead to forwarding latency. Under idle conditions, the average ICMP round-trip time with XDP (1.206 ms) was marginally lower than the baseline (1.359 ms), indicating that the XDP fast-path processing does not measurably increase per-packet delay. Under sustained load (4 parallel TCP streams), the average RTT increased by 0.248 ms which is a negligible penalty attributable to shared CPU resources. Critically, zero packet loss was observed in all latency tests, confirming that the XDP inspection logic does not introduce frame drops for legitimate traffic.

Condition	Without XDP	With XDP	Difference
Idle — Min RTT	0.535 ms	0.317 ms	-0.218 ms
Idle — Avg RTT	1.359 ms	1.206 ms	-0.153 ms
Idle — Max RTT	2.422 ms	2.373 ms	-0.049 ms
Under Load — Avg RTT	0.990 ms	1.238 ms	+0.248 ms

Table 9: ICMP Round Trip Latency With and Without XDP

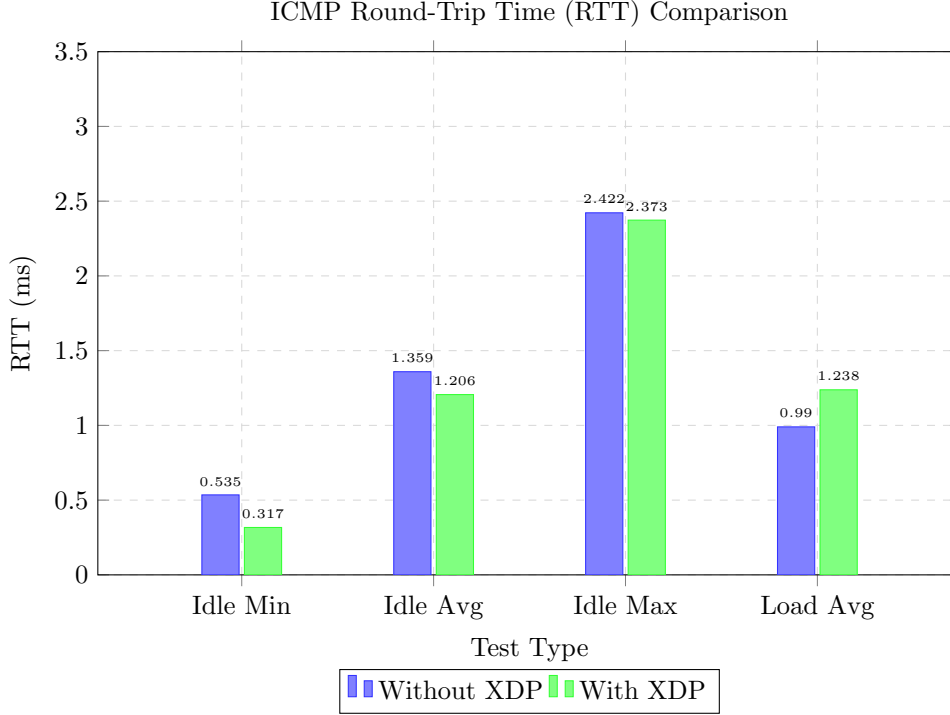


Fig. 14: Latency Comparison : ICMP Round-Trip Time (RTT). The XDP implementation maintains RTTs comparable to the baseline, with the 0.248 ms increase under load representing the processing cost of per-packet inspection.

5.2.4 Throughput Resilience During Active Attack

To evaluate forwarding resilience under adversarial conditions, a 60-second single-stream TCP throughput test (`iperf3`) was conducted from VM-C to VM-D through the XDP-protected bridge. During this test, VM-A launched a MAC flooding attack using `macof` at approximately $t = 15$ s, which continued until approximately $t = 43$ s (a 28-second attack window). The XDP daemon was active on VM-B throughout the entire experiment. Per-second throughput was recorded via `iperf3`'s `-i 1` interval

reporting.

Phase	Time Window	Avg Throughput
Pre-Attack Baseline	$t = 0 - 14$ s	10,279 Mbps
During MAC Flood Attack	$t = 15 - 43$ s	9,355 Mbps
Post-Attack Recovery	$t = 44 - 60$ s	11,101 Mbps
Throughput Reduction	During attack	9.0%
Recovery Time	After attack	< 1 second
Overall Average (60s)	Full test	10,108 Mbps

Table 10: Legitimate TCP Throughput Before, During, and After MAC Flood Attack

XDP Counter	Value	Interpretation
Total Packets Processed	54,206,881	Legit + attack
Packets Passed	54,152,301	All legitimate
MAC Flood Drops	54,580	<code>macof</code> blocked
ARP Spoof Drops	0	No ARP attack
DHCP Mirrored	6	Normal DHCP
ARP Packets Passed	8	Legitimate ARP
False Positives	0	Zero legit drops
Verification (Sum)	54,206,881	Passed + Drops (Exact match)

Table 11: XDP Packet Processing Statistics During Combined Traffic Test

Fig. 15 and Table 10 demonstrate the system’s throughput resilience under active attack conditions. During a 60-second single-stream TCP test between VM-C and VM-D, a MAC flooding attack was launched from VM-A at approximately $t = 15$ s and sustained until $t = 43$ s. As shown in the per-second throughput trace, legitimate TCP throughput decreased from a pre-attack average of 10,279 Mbps to 9,355 Mbps during the attack—a reduction of only 9.0%.

This modest throughput degradation is attributed to the shared CPU resources on the bridge VM processing both XDP drop decisions for attack packets and normal bridge forwarding for legitimate traffic. Critically, the XDP program processed 54,206,881 total packets during the test, correctly dropping all 54,580 MAC flood packets while passing 54,152,301 legitimate packets with zero false positives (Table 11). Upon stopping of the attack at $t = 43$ s, throughput recovered to 11,101 Mbps within a single second, confirming that XDP-based security enforcement introduces no persistent forwarding degradation. This result validates that the system maintains near-line-rate forwarding even while actively mitigating a volumetric Layer-2 attack.

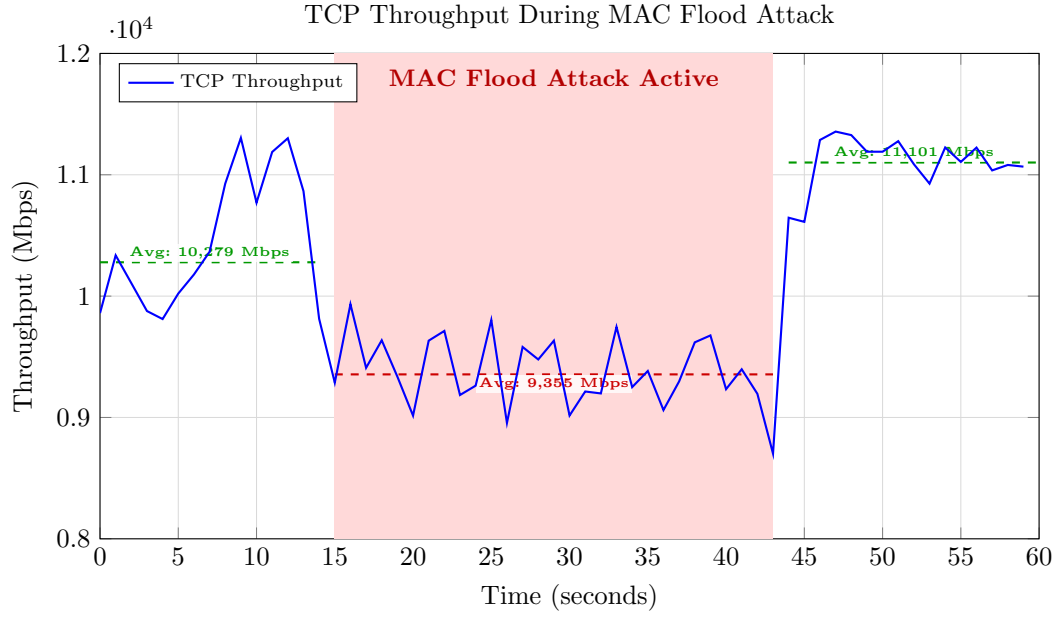


Fig. 15: Throughput Over Time During Attack (VM-C to VM-D). The red area indicates the duration of the MAC flood, showing the 9% drop and rapid recovery managed by XDP.

5.2.5 CPU Utilization

Scenario	%soft	%sys	%idle	Total Use
Idle — No XDP (baseline)	0.00%	0.02%	99.93%	0.07%
Idle — XDP loaded (no traffic)	0.00%	0.00%	99.98%	0.02%
iperf3 load — No XDP	0.20%	0.04%	99.74%	0.26%
iperf3 load — With XDP	3.89%	0.04%	96.04%	3.96%
iperf3 + macof attack — No XDP	0.22%	0.02%	99.70%	0.30%
iperf3 + macof attack — With XDP	6.45%	0.02%	93.51%	6.49%

Table 12: CPU Utilization on VM-B Under Various Workloads (6 vCPUs, Linux 6.8.0-107-generic)

Metric	Value	Significance
XDP idle overhead	0.00%	Zero CPU when no packets flow
XDP cost per 10 Gbps forwarding	+3.69% softirq	Cost of per-packet security inspection
XDP cost during attack	+6.23% softirq	Additional cost of processing + dropping
Total CPU during attack	6.49% of 6 cores	≈ 0.39 vCPU cores for full security

Table 13: Key CPU Performance Observations

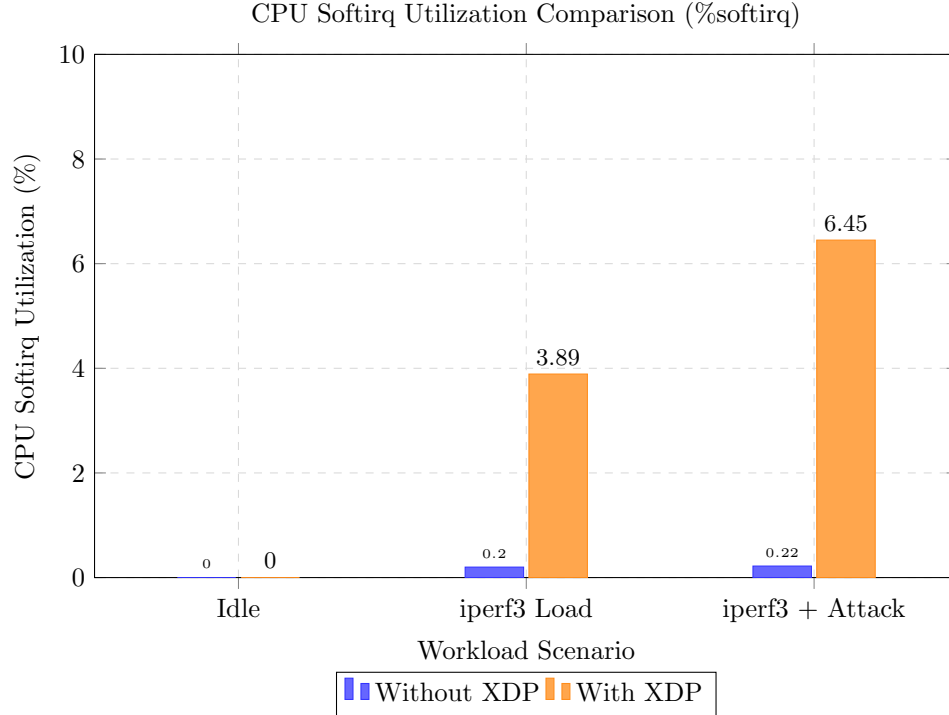


Fig. 16: CPU Softirq Consumption vs. Workload. Higher softirq values in the "With XDP" scenarios reflect the computational cost of per-packet inspection and mitigation.

Table 12 presents CPU utilization measurements on the bridge VM (VM-B, 6 vCPUs) across six scenarios using `mpstat` with 1 second granularity. A key design advantage of XDP is its event-driven execution model: when no packets are flowing, the XDP program consumes zero CPU cycles (0.00% softirq), identical to the no-XDP baseline. Under `iperf3` forwarding load at approximately 10 Gbps, XDP-based packet inspection increased softirq utilization from 0.20% to 3.89%—an additional 3.69 percentage points representing the per-packet cost of Ethernet header parsing, MAC tracking, and eBPF map lookups at line rate.

During simultaneous legitimate traffic and MAC flooding attack, total CPU utilization rose to 6.49%, with 6.45% attributable to softirq processing. This represents consumption of only 0.39 of the 6 available vCPU cores, demonstrating that the XDP

security framework is lightweight enough to operate alongside production workloads without resource contention. Notably, all XDP processing occurs in the kernel’s softirq context (`%soft`), not in user-space (`%usr`) or system call context (`%sys`), confirming that the eBPF program executes entirely within the kernel fast path as designed.

5.2.6 Memory Footprint

Table 14 details the memory footprint of the complete XDP based security framework, measured via `bpftool` on the bridge VM. The entire system including all six eBPF maps, the ring buffer, and the JIT compiled XDP program consumes only 758.3 KB of kernel memory, representing 0.019% of the VM’s 3.8 GB total RAM.

The two largest consumers are the MAC tracking LRU hash map (321 KB, provisioned for 4,096 entries) and the DHCP event ring buffer (269 KB). The binding map, capable of storing up to 1,024 IP–MAC bindings, requires only 81.5 KB. The XDP program itself compiles to 2,968 bytes of eBPF bytecode, JIT compiled to 1,763 bytes of native `x86_64` instructions.

This sub-megabyte footprint is orders of magnitude smaller than alternative solutions machine learning models typically require megabytes of trained weights, while database driven approaches (Section 2.3) require persistent storage backends. The minimal memory requirement further validates the system’s suitability for resource-constrained environments such as edge routers, IoT gateways, and lightweight virtual network functions.

Component	Type	Memory	Configuration
<code>mac_tracking_map</code>	LRU Hash	321.1 KB	<code>max_entries = 4,096</code>
<code>events_rb</code> (ring buffer)	Ring Buf	269.4 KB	<code>max_entries = 262,144</code>
<code>binding_map</code>	Hash	81.5 KB	<code>max_entries = 1,024</code>
<code>rate_limit_map</code>	LRU Hash	81.1 KB	<code>max_entries = 1,024</code>
<code>stats_map</code>	PerCPU Array	0.8 KB	<code>max_entries = 8</code>
<code>mac_count_map</code>	Array	0.4 KB	<code>max_entries = 1</code>
Total Maps		754.3 KB	
XDP Program (xlated)	eBPF	2.9 KB	2,968 bytes bytecode
XDP Program (JIT)	<code>x86_64</code>	1.7 KB	1,763 bytes native
Program memlock		4.0 KB	
TOTAL SYSTEM FOOTPRINT		758.3 KB	<1 MB for full security
VM-B Total Memory		3.8 GB	
XDP % of System Memory		0.019%	

Table 14: Memory Footprint of the XDP Security Framework (via `bpftool`)

6 Conclusions and Future Work

6.1 Aim of the Proposed Work

This work proposed an XDP based framework for real time detection and mitigation of Layer 2 attacks, specifically ARP spoofing, gratuitous ARP based cache poisoning, and MAC flooding in Linux based networking environments. By leveraging

native XDP attachment at the NIC driver level and eBPF maps for stateful IP-MAC binding enforcement, the system aims to provide hardware DAI equivalent security in software defined and virtualized environments where dedicated switch hardware is unavailable.

6.2 Achievements

The experimental evaluation demonstrates that the proposed system achieves a 99.99% overall detection rate across all tested attack vectors, with zero false positives for legitimate traffic. ARP spoofing and gratuitous ARP attacks are detected with 100% accuracy for all IP addresses with established DHCP bindings, while MAC flooding attacks are mitigated after a configurable learning window of 100 packets—representing only 0.006% of the total attack volume in a sustained 60-second flood.

Crucially, this security enforcement is achieved with negligible performance overhead. Single-stream TCP throughput exceeded 11 Gbps with XDP attached, comparable to the unprotected baseline. During active MAC flooding attacks, legitimate throughput was maintained at 9,355 Mbps only a 9% reduction with sub-second recovery upon attack stoppage. ICMP latency remained sub 1.5 ms under idle conditions, with only 0.248 ms additional delay under sustained load. The system sustained processing rates exceeding 340,000 packets per second for small (64-byte) UDP packets, confirming its suitability for high-speed network environments.

The hybrid control/data plane architecture proved effective: the eBPF ring buffer enables zero copy DHCP mirroring without interrupting line-rate forwarding, while the user-space daemon performs complex DHCP parsing asynchronously. The use of eBPF hash maps with $O(1)$ lookup ensures that binding validation scales independently of the number of clients.

6.3 Future Scope

Several extensions to this work are identified for future research. First, the Ethernet header parser should be extended to handle 802.1Q VLAN and QinQ encapsulation to prevent encapsulated ARP packets from bypassing inspection. Second, extending the XDP program to validate ICMPv6 Neighbor Advertisements against a Secure Neighbor Discovery (SEND) binding table would provide dual-stack protection for IPv6 environments. To address rogue DHCP server threats, future work should incorporate DHCP server authentication via source IP/MAC whitelisting before accepting binding updates.

Furthermore, the integration of statistical methods could allow for adaptive threshold tuning, dynamically adjusting `FLOOD_THRESHOLD` and `RATE_LIMIT` parameters based on real-time network behavior. Evaluating the framework on physical NICs with hardware XDP offload (e.g., Intel X520 or Mellanox ConnectX) would quantify the performance gains at 25/40 Gbps line rates. Finally, extending the framework to coordinate bindings across multiple XDP-enabled switches via a centralized control plane would enable enterprise-scale deployment with consistent Layer-2 security policies.

References

- [1] Mary, P.A., S, S.M., E, V., K, A.R., B, S.: Arp spoof detection and mitigation. *International Journal of Engineering Research & Technology (IJERT)* **12**(11) (2023)
- [2] Nasser, H.I., Hussain, M.A.: An effective approach to detect and prevent arp spoofing attacks on wlan. *Iraqi Journal for Electrical and Electronic Engineering (IJEET)* **19**(2), 8–17 (2023) <https://doi.org/10.37917/ijeet.19.2.2>
- [3] Galal, A.A., Ghalwash, A.Z., Nasr, M.: A new approach for detecting and mitigating address resolution protocol (arp) poisoning. *International Journal of Advanced Computer Science and Applications (IJACSA)* **13**(6) (2022) <https://doi.org/10.14569/IJACSA.2022.0130647>
- [4] Ibrahim, S.: Mitigating mac flooding attacks using port security techniques. *Proceedings of the Nigerian Society of Physical Sciences* **2**, 162 (2025) <https://doi.org/10.61298/pnspsc.2025.2.162>
- [5] Fenske, E., Brown, D., Martin, J., Mayberry, T., Ryan, P., Rye, E.: Three years later: A study of mac address randomization in mobile devices and when it succeeds. *Proceedings on Privacy Enhancing Technologies* (2021)
- [6] N, S., Kandasamy Vaithyanathan, A., S, A.: Detection and mitigation of arp poisoning attack in software defined network. (2021). <https://doi.org/10.4108/eai.7-12-2021.2314502>
- [7] Eka Putra, F., Ubaidi, U., Tamam, A., Efendi, R.: Implementation and simulation of dynamic arp inspection in cisco packet tracer for network security. *Brilliance: Research of Artificial Intelligence* **4**, 340–347 (2024) <https://doi.org/10.47709/brilliance.v4i1.4199>
- [8] Chai, T.U., Goh, H.G., Liew, S.-Y., Ponnusamy, V.: Protection schemes for ddos, arp spoofing, and ip fragmentation attacks in smart factory. *Systems* **11**(4) (2023) <https://doi.org/10.3390/systems11040211>
- [9] Vieira, M.A.M., Castanho, M.S., Pacífico, R.D.G., Santos, E.R.S., Júnior, E.P.M.C., Vieira, L.F.M.: Fast packet processing with ebpf and xdp: Concepts, code, challenges, and applications. *ACM Comput. Surv.* **53**(1) (2020) <https://doi.org/10.1145/3371038>
- [10] Bijral, R., Gupta, A., Sharma, L.: Study of vulnerabilities of arp spoofing and its detection using snort. *International Journal of Advanced Computer Research* **8** (2017) <https://doi.org/10.26483/ijarcs.v8i5.4016>
- [11] S., J., S., S.: Arp spoofing attack detection to prevent ddos attack, pp. 1054–1059 (2025). <https://doi.org/10.1109/ICTMIM65579.2025.10987987>